

Blockchain Mini Project Report

Decentralized Voting System

Submitted fulfilment of the requirements of the
Blockchain Mini Project Semester VII in degree of
Bachelor of Computer Engineering

BY

Riya Jha (47)

Rugved Khandake (60)

Guide

Asst. Prof. Manasi Churi



Department Of Computer Engineering
Shree L. R. Tiwari College Of Engineering
Kanakia Park, Mira Road (E), Thane -401 107, Maharashtra.

2026-27

Blockchain Mini Project Report Approval

This project report entitled **“Decentralized Voting System”** by **Ms. Riya Jha (47)** and **Mr. Rugved Khandake (60)** is approved for the Mini Project Semester VII project work of B.E CMPN at Shree L.R.Tiwari College of Engineering, in the academic year 2026-2027.

Internal Examiner

Declaration

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Signature
Ms. Riya Jha (47)

Signature
Mr. Rugved Khandake (60)

Date:

Place:

Table of Contents

1. Introduction
2. System Architecture
3. Implementation
4. Results
5. Conclusion
6. References

1. Introduction

Traditional voting systems often rely on centralized authorities for voter rolls, ballot counting, and result publication. Centralization can raise concerns about transparency, single points of failure, and the difficulty of independently verifying that each eligible voter cast exactly one ballot and that tallies were not altered after the fact.

Blockchain technology provides an immutable, shared ledger and programmable rules via smart contracts. An election contract can record candidate lists, prevent double voting with a `hasVoted` mapping, accept votes from any address while the poll is open, and publish a winner only after the owner ends the election. Events such as `VoteCast` and `ElectionEnded` give an auditable trail of participation.

This mini-project implements a Decentralized Voting System as a local MVP for the Blockchain laboratory (Semester VII). The contract owner creates an election with a title and candidate names. Any wallet may vote once for one candidate. The owner ends the election, after which the winner is readable on-chain. The stack uses Hardhat for local development, Solidity for on-chain logic, and a lightweight HTML/CSS/JavaScript frontend with ethers.js v6.

Objectives

1. Design a Solidity smart contract for creating elections, casting one vote per address, and ending the poll.
2. Emit `VoteCast` and `ElectionEnded` events and expose candidate tallies and `getWinner` after end.
3. Provide unit tests and a runnable local demo (Hardhat node + MetaMask-compatible UI).
4. Document architecture, implementation, and results for academic evaluation.

Scope (MVP)

- Local Hardhat network only; no public-chain deployment.
- One election per contract deployment; no voter identity KYC beyond wallet address.
- No token-weighted or quadratic voting; one address, one vote.

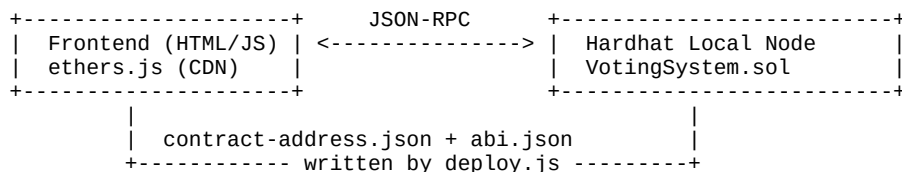
2. System Architecture

The system involves two primary actors. The owner (contract deployer) creates an election with a title and at least two candidate names, and may later end the election. A voter connects a wallet, selects one candidate while the election is open, and submits a vote transaction. The contract rejects a second vote from the same address and rejects votes after the election has ended.

The system is organized in three layers:

- **Smart contract layer** — VotingSystem.sol stores election metadata, Candidate records, hasVoted mapping, and emits ElectionCreated / VoteCast / ElectionEnded.
- **Tooling layer** — Hardhat compiles, tests, and deploys the contract. The deploy script seeds a sample election and writes the address and ABI for the UI.
- **Presentation layer** — A vanilla single-page app loads the ABI and address, connects a wallet (or runs read-only against the local node), lists candidates with tallies, submits votes, and lets the owner end the election.

Logical architecture (local MVP):



On-chain data and key flows

Each Candidate struct holds id, name, and voteCount. createElection (owner-only) sets electionTitle, marks electionCreated, and stores candidates. vote checks that the election is open, the sender has not voted, and the candidate id is valid, then sets hasVoted[msg.sender] and increments the tally. endElection (owner-only) sets electionEnded and emits ElectionEnded with the winning candidate. getWinner is available only after end; ties prefer the lowest candidate id among leaders.

3. Implementation

The core contract is `contracts/VotingSystem.sol` (Solidity ^0.8.20). Custom errors provide clear reverts (`NotOwner`, `AlreadyVoted`, `ElectionAlreadyEnded`, `InvalidCandidate`, and others). View helpers include `getCandidate`, `getAllCandidates`, `candidateCount`, `hasVoted`, and `getWinner` (post-end).

Key Solidity functions:

```
function createElection(string calldata title, string[] calldata names)
    external onlyOwner
{
    if (electionCreated) revert ElectionAlreadyCreated();
    if (bytes(title).length == 0) revert EmptyTitle();
    if (names.length < 2) revert NeedCandidates();
    electionTitle = title;
    electionCreated = true;
    for (uint256 i = 0; i < names.length; i++) {
        candidateCount++;
        _candidates[candidateCount] = Candidate({
            id: candidateCount, name: names[i], voteCount: 0
        });
    }
    emit ElectionCreated(title, candidateCount);
}

function vote(uint256 candidateId) external {
    if (!electionCreated) revert ElectionNotCreated();
    if (electionEnded) revert ElectionAlreadyEnded();
    if (hasVoted[msg.sender]) revert AlreadyVoted();
    if (candidateId == 0 || candidateId > candidateCount)
        revert InvalidCandidate();
    hasVoted[msg.sender] = true;
    _candidates[candidateId].voteCount += 1;
    emit VoteCast(msg.sender, candidateId);
}

function endElection() external onlyOwner {
    if (!electionCreated) revert ElectionNotCreated();
    if (electionEnded) revert ElectionAlreadyEnded();
    electionEnded = true;
    (uint256 winnerId, string memory winnerName, uint256 winnerVotes)
        = _computeWinner();
    emit ElectionEnded(winnerId, winnerName, winnerVotes);
}
```

Hardhat project layout includes `hardhat.config.js` (Solidity 0.8.20, localhost), `scripts/deploy.js` (deploys, seeds a sample election with three candidates, writes `frontend/contract-address.json` and `frontend/abi.json`), `scripts/smoke.js` (vote + end + winner), and `test/VotingSystem.js` (Mocha/Chai coverage of create, vote, double-vote revert, end, winner, cannot vote after end, non-owner create, and `getWinner` before end). The frontend under `frontend/` uses `index.html`, `styles.css`, and `app.js` with `ethers.js` v6 from CDN.

Technologies Used

- **Solidity ^0.8.20** — Smart contract language for VotingSystem.sol.
- **Hardhat** — Compile, unit test, local blockchain node, and deployment.
- **ethers.js v6** — Contract interaction in tests and the browser frontend.
- **Node.js** — Runtime for Hardhat tooling and npm scripts.
- **Vanilla HTML / CSS / JavaScript** — Voting UI without a heavy framework.
- **MetaMask (optional)** — Injected wallet for interactive vote and owner demos.

4. Results

The project was verified on a local Hardhat environment. Compilation succeeds with `npx hardhat compile`. All eight unit tests pass under `npx hardhat test` (create election, vote, double-vote revert, end election, winner, cannot vote after end, non-owner create, and getWinner before end). A smoke run with a live Hardhat node successfully casts a vote, ends the election, and confirms the expected winner.

After deployment, the static frontend shows the seeded election title and candidates with live vote counts on the local network. Connecting a Hardhat account allows voting while the poll is open. The owner account can end the election; the UI then displays the winner. Read-only JsonRpcProvider mode works without MetaMask for demonstration of tallies.

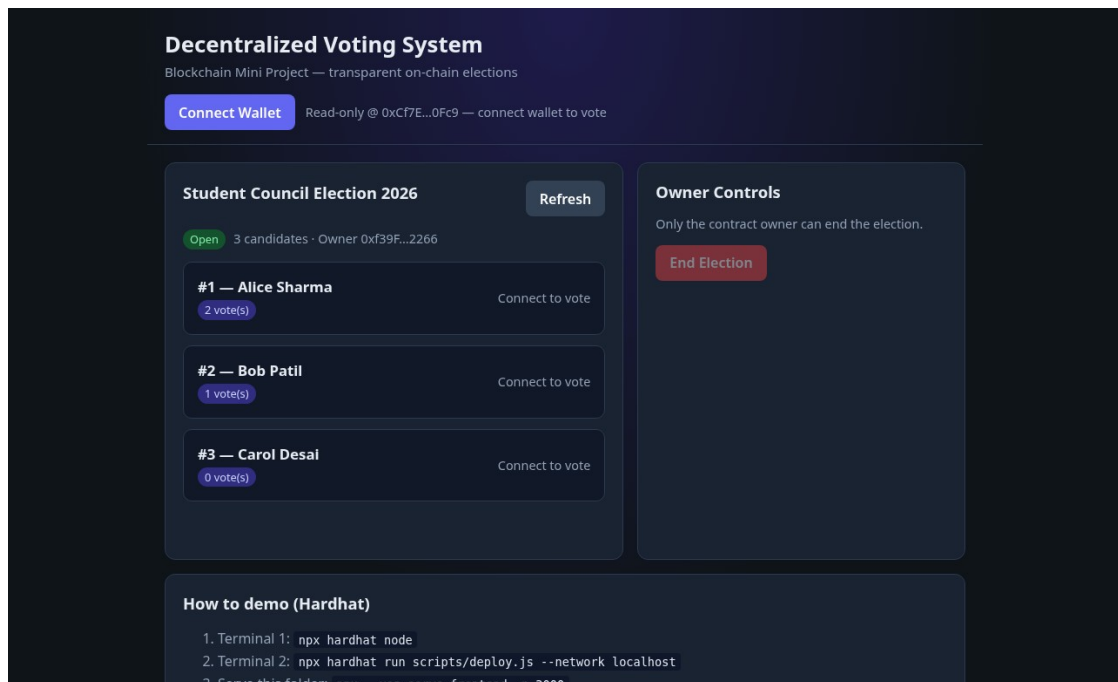


Fig. 1: Open election with candidates and live tallies

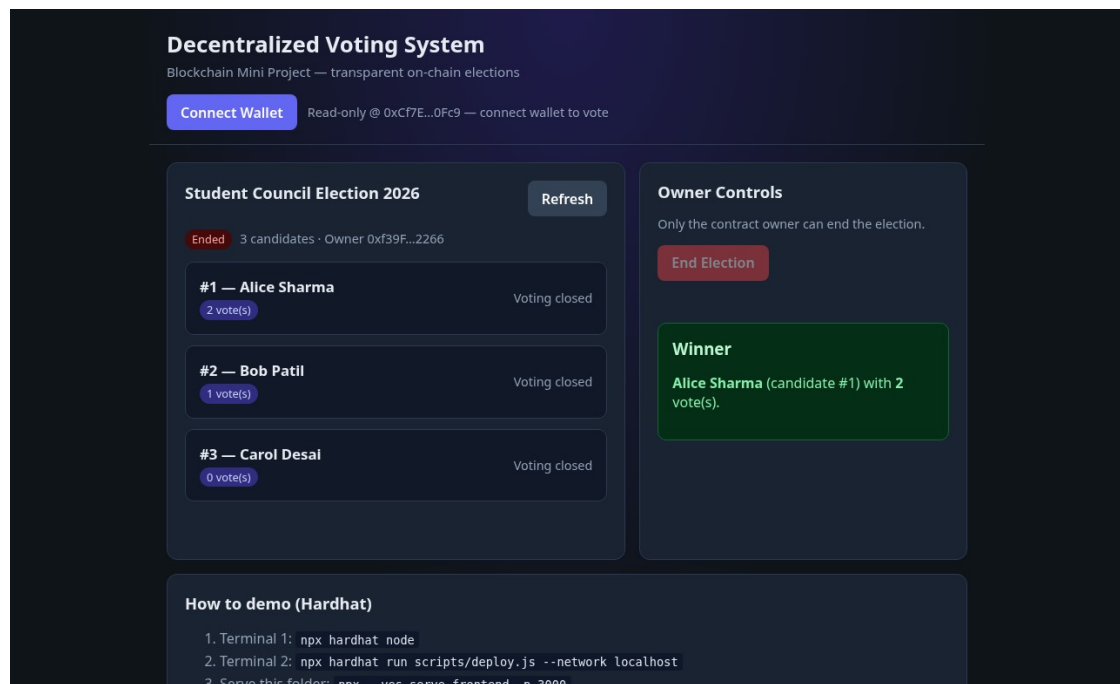


Fig. 2: Election ended — winner displayed

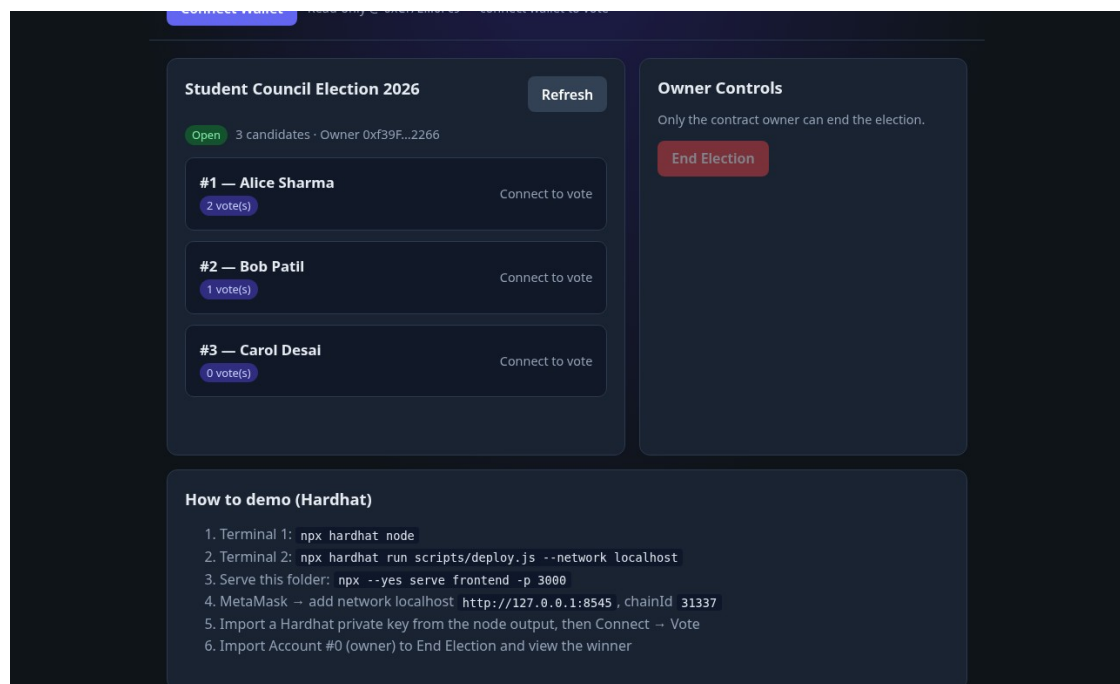


Fig. 3: Demo help panel (Hardhat steps)

Summary: 8/8 Hardhat tests passed; smoke vote + end + winner succeeded; the screenshots above confirm the local voting demo path used for laboratory evaluation.

5. Conclusion

This mini-project delivered a working Decentralized Voting System MVP suitable for a Blockchain laboratory demonstration. A Solidity contract manages election creation, one-vote-per-address casting, election closure, and winner publication. Hardhat provides compilation, automated tests (8/8 passing), and a local chain. A simple web UI lets users interact through a wallet or in read-only mode against the deployed contract address.

The design deliberately stays small: no public-chain deployment, no off-chain voter registry, and a single election per deployment. Future work may include multiple concurrent elections, commit-reveal ballots for privacy, Merkle-proof voter eligibility, or time-locked voting windows. Within the stated scope, the happy path—from create to vote to end to winner—is implemented, tested, and documented.

6. References

1. Ethereum Foundation, “Ethereum Developer Documentation,” <https://ethereum.org/developers/docs/> (accessed 2026).
2. Solidity Team, “Solidity Documentation,” <https://docs.soliditylang.org/> (accessed 2026).
3. Nomic Foundation, “Hardhat Documentation,” <https://hardhat.org/docs> (accessed 2026).
4. ethers.js, “ethers.js v6 Documentation,” <https://docs.ethers.org/v6/> (accessed 2026).
5. Ethereum Foundation, “Smart Contract Security Best Practices — Access Control,” applied via onlyOwner for create/end.
6. Shree L. R. Tiwari College Of Engineering, Department of Computer Engineering — Blockchain Mini Project guidelines (internal).